

Guía de Actividades Práctico-Experimentales Nro. 001

1. Datos Generales

Asignatura	Sistemas distribuidos
Ciclo	6to
Unidad	Envío de mensajes distribuido
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	Uno
Título de la Práctica	Análisis Colaborativo de las Arquitecturas Principales en un Sistema Distribuido
Nombre del Docente	José O. Guamán Q.
Fecha	29 de septiembre de 2026 02 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica:

Analizar de manera colaborativa las arquitecturas fundamentales de los sistemas distribuidos (cliente-servidor, peer-to-peer, multicapa y microservicios), identificando sus características, ventajas, desventajas y casos de uso, mediante trabajo en equipo y el diseño de un caso de estudio

- Identificar las características estructurales y funcionales de las arquitecturas cliente-servidor, peer-to-peer, multicapa (n-tier), y microservicios en sistemas distribuidos.
- Comparar las ventajas, desventajas, escalabilidad y tolerancia a fallos de cada arquitectura mediante el llenado colaborativo de una matriz de análisis.
- Diseñar el diagrama de un caso de estudio propuesto aplicando la arquitectura distribuida más adecuada, justificando técnicamente la elección del equipo

3. Materiales y reactivos:

Diapositivas, y enunciados del problema:

- Hardware:
 - Computadora personal con conexión a internet.
- Software:
 - Procesador de texto.

4. Equipos y herramientas

Computadora personal con conexión a internet

Guía de práctica.

Plantilla de matriz comparativa

5. Procedimiento / Metodología

Fase 0 (10 min). Formación de equipos de 4 (asignación del docente).

Fase 1 – Investigación (50 min). Cada integrante investiga su arquitectura y completa su sección de la matriz (estructura, ventajas, desventajas, escalabilidad, ejemplos). El equipo elabora el diagrama de su arquitectura en draw.io.

Fase 2 – Socialización (30 min). Cada equipo expone 5 min. El resto completa la matriz comparativa con la información expuesta (una sola matriz, grupal).

Fase 3 – Caso de estudio (55 min). Cada equipo selecciona la arquitectura más adecuada para el enunciado, la justifica con mínimo 3 argumentos técnicos, elabora el diagrama de la solución e identifica dónde se aplican transparencia, escalabilidad y tolerancia a fallos.

Fase 4 – Reporte y entrega (25 min). Cada estudiante redacta sus conclusiones individuales; el equipo compila la matriz final y los diagramas. Entrega en la plataforma indicada.

Criterio	Cliente -Servidor	Peer-to -Peer	Multicapa	Microservicios
Estructura	La estructura está conformada por dos nodos, tanto como el nodo cliente y el nodo servidor [1]. El nodo cliente es aquel que se encarga de interactuar con el usuario, mientras tanto que el nodo servidor este se encarga de administrar la información de los clientes [1].	La estructura se basa en una red de computadoras donde los nodos también llamados peers actúan tanto como clientes y servidores.[2] Los nodos se dividen en peers simples donde los propietarios de los recursos tienen una conectividad intermitente y los super peers que cuentan con una alta capacidad y una dirección IP pública que ayuda en la localización de recursos.[2] Se puede organizar en modelos centralizados, no	La estructura de esta arquitectura usualmente organiza en 3 capas [1]. La primera capa es la de presentación o usuario en donde se muestra la información y captura acciones [1]. Prosigue la capa de negocio en donde están todas las reglas de negocio y validaciones, flujos de trabajo y procesamiento de datos [1].	La aplicación se divide en servicios pequeños que trabajan juntos. Cada microservicio se encarga de una función concreta del negocio y puede ejecutarse de manera independiente. Cuando necesita información o ayuda de otro servicio, se comunica con él mediante mensajes o una API. Así, aunque cada servicio tiene su propia responsabilidad, todos colaboran para que la aplicación funcione. Por eso, es importante definir bien qué hace cada uno y cómo se

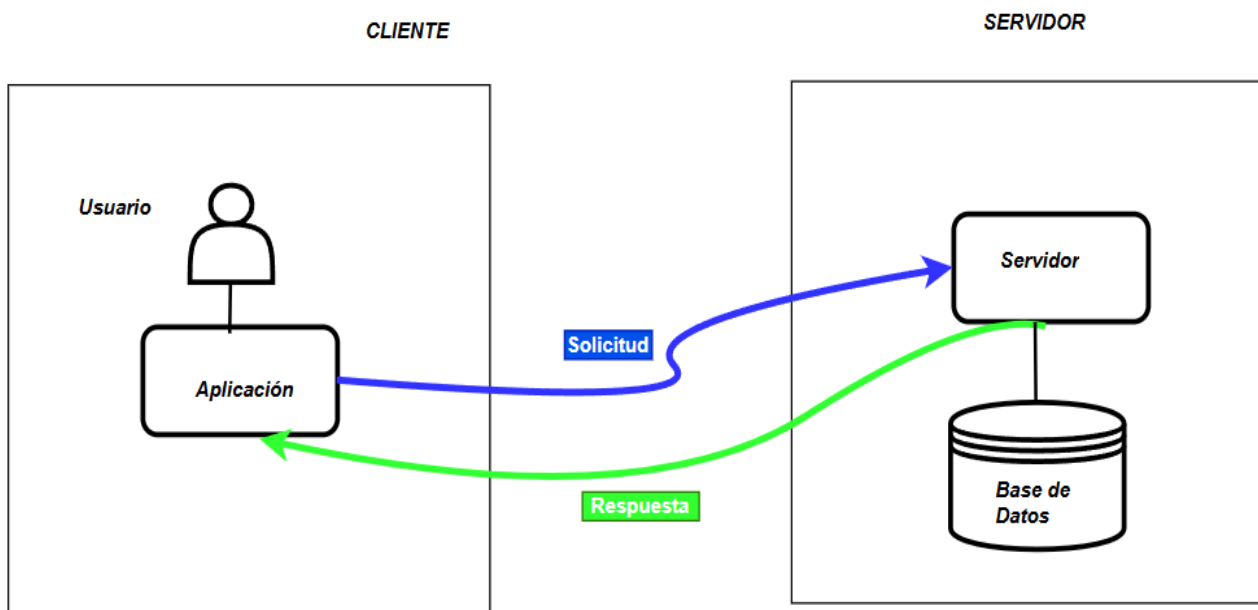
		estructurados, estructurados y híbridos. [2]	En tercer lugar está la capa de acceso a datos que encapsula las operaciones de lectura y escritura en los medios de almacenamiento y aísla la lógica del negocio de la base de datos [1]. Finalmente se encuentra la base de datos que es el almacenamiento físico de la información de forma estructurada y persistente [1].	comunican entre sí [4], [5], [6].
Ventajas	Esta arquitectura nos proporciona las siguientes ventajas: Organización de las funciones entre cliente y servidor. Centralización de la información y los recursos. Simplificación de la administración y mantenimiento del sistema. Facilita el control de acceso y la seguridad de la información.	Al ser una arquitectura descentralizada las ventajas que posee son varias tales como: una mejora en la escalabilidad al permitir a las redes peer-to-peer albergar un número ilimitado de nodos sin afectar el tiempo de respuesta de la red ; permite a los usuarios utilizar un overlay de comunicación sin repercusiones legales o de otro tipo como ataques a la intimidad manteniendo el anonimato ; la propiedad compartida la cual es una de las premisas del P2P reduce el coste de gestión de los sistemas y contenidos así como su mantenimiento [2]	Esta arquitectura ofrece varias ventajas debido a su organización jerárquica. Esto permite la separación de responsabilidades, permite un desarrollo paralelo y facilita la mantenibilidad y modularidad de los sistemas que se implementan. Además integran un sistema de seguridad integrado al separar la capa de presentación de la base de datos. Finalmente, permite la reutilización de código desde una misma lógica de negocio hacia múltiples interfaces [1].	Se puede modificar, corregir y desplegar un servicio sin tener que desplegar nuevamente toda la aplicación. Como cada servicio se concentra en una función, resulta más fácil entender su código y darle mantenimiento. También permite que distintos equipos se encarguen de servicios específicos y trabajen con mayor autonomía. Además, si la arquitectura está diseñada para aislar los fallos, un problema en un servicio no tiene por qué detener a todos los demás [4], [5].
Desventajas	También dentro de esta arquitectura nos encontramos las siguientes desventajas: Problemas de escalabilidad cuando aumenta considerablemente el número de clientes. Mayor consumo de recursos en el	Sin embargo al ser una arquitectura descentralizada las desventajas que presenta son: una alta volatilidad en los nodos que al conectarse y desconectarse de forma continua incrementa la posibilidad de	Por otra parte, también tiene puntos débiles. Entre ellos se destacan la alta complejidad de gestión y orquestación de los múltiples servicios e interfaces, además que a veces simples cambios en	Tener varios servicios separados también hace más complicado el funcionamiento de la aplicación. Si una operación necesita comunicarse con varios servicios, esas llamadas por red pueden causar demoras. Cuando los

	servidor debido a la cantidad de solicitudes. Dependencia del servidor, ya que una falla puede afectar a los clientes. Dependencia de la conexión de red para la comunicación entre cliente y servidor.	fallos en el sistema y provoca la pérdida momentánea de disponibilidad de los recursos, una sobrecarga en el ancho de banda y en los tiempos de respuesta causada por la saturación de tráfico, una mayor complejidad para garantizar la seguridad contra nodos maliciosos y para gestionar de forma eficiente recursos mutables que cambian constantemente de estado a lo largo del tiempo como los ciclos de CPU [2].	los datos pueden requerir actualización del esquema de la base de datos, además cada llamada puede atravesar múltiples capas lo que provoca un aumento de latencia para las consultas [1].	datos están repartidos entre servicios, se necesitan mecanismos adicionales para mantenerlos consistentes. Encontrar un error también puede tomar más tiempo, porque hay que seguir el recorrido de la solicitud por distintos servicios. Por eso, las pruebas deben revisar tanto cada servicio por separado como la comunicación y el funcionamiento conjunto de todos ellos [4], [5], [6].
Escalabilidad	Esta arquitectura es escalable mediante el aumento de los recursos del servidor. También es posible incorporar varios servidores y distribuir las solicitudes entre ellos para atender un mayor número de clientes.	Alta capacidad de escalado global permitiendo albergar potencialmente un número ilimitado de nodos a nivel mundial. [2]	Tiene factores que favorecen la escalabilidad, como la distribución de carga de trabajo entre múltiples nodos o servidores y la eliminación de puntos únicos de fallo que permiten añadir más nodos sin degradar el rendimiento global. Por otra parte ha implementado mecanismos modernos de escalado como los microservicios y contenedores que permiten escalar componentes según la demanda. Sin embargo, hay que considerar que a medida que crece el número de nodos, la sincronización y gestión de comunicaciones pueden verse sobresaturados [1].	Se puede aumentar la capacidad de cada servicio según la cantidad de solicitudes que recibe. Si una función tiene más demanda que las demás, se añaden más instancias del servicio encargado de atenderla, sin tener que ampliar toda la aplicación. Esto ayuda a aprovechar mejor el hardware, porque los recursos se concentran donde hacen falta. Sin embargo, cuando aumenta el número de servicios, también pueden crecer la infraestructura y el esfuerzo necesarios para administrarlos [5].
Ejemplos reales	Como ejemplos de sistemas en donde se implementa esta	Ejemplos de uso real de esta arquitectura son en	Uno de los sistemas actuales más complejos y	Netflix y SoundCloud aparecen en los

arquitectura seria en un sistema Universitario, ya que el estudiante utiliza un navegador como cliente y el servidor de la universidad procesa las consultas, además esta arquitectura se puede presentar también en tiendas en línea como Amazon.	cuanto a la compartición de archivos: BitTorrent, Kazaa, Gnutella, plataformas académicas como DisCoP, CompuP2P y plataformas de colaboración telefónica: Skype y Groove. [2]	delicados que manejan multicapa son las aplicaciones bancarias. Esto debido a que manejan una capa monolítica y tuvo dificultades de escalabilidad y mantenimiento. Para resolverlas, fue necesario pasar a nuevos microservicios para validar la lógica del negocio, hacer otras. En el caso de uso de mapeadores de datos para acciones como ejecutar transacciones seguras y para la base de datos utilizan un sistema relacional como Oracle o PostgreSQL.	artículos como empresas que adoptaron microservicios [4]. SoundCloud empezó con una aplicación y tuvo dificultades de escalabilidad y mantenimiento. Para resolverlas, fue necesario pasar a nuevos microservicios para validar la lógica del negocio, hacer otras. En el caso de uso de mapeadores de datos para acciones como ejecutar transacciones seguras y para la base de datos utilizan un sistema relacional como Oracle o PostgreSQL.
--	---	--	---

1. Cliente servidor

Figura 1. Ejemplo básico una arquitectura Cliente Servidor [Elaboración Propia].



2. Peer-to-Peer

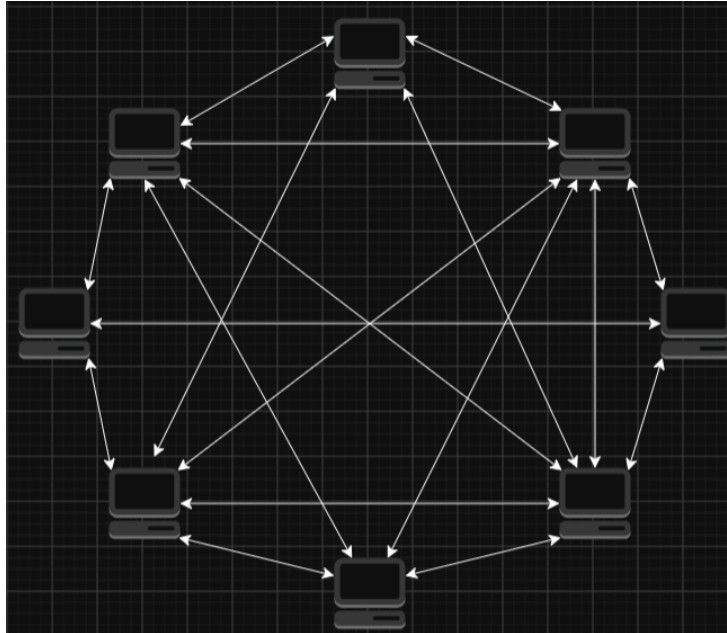


Figura 2. Ejemplo básico una arquitectura Peer-to-Peer [Elaboración Propia].

3. Multicapa

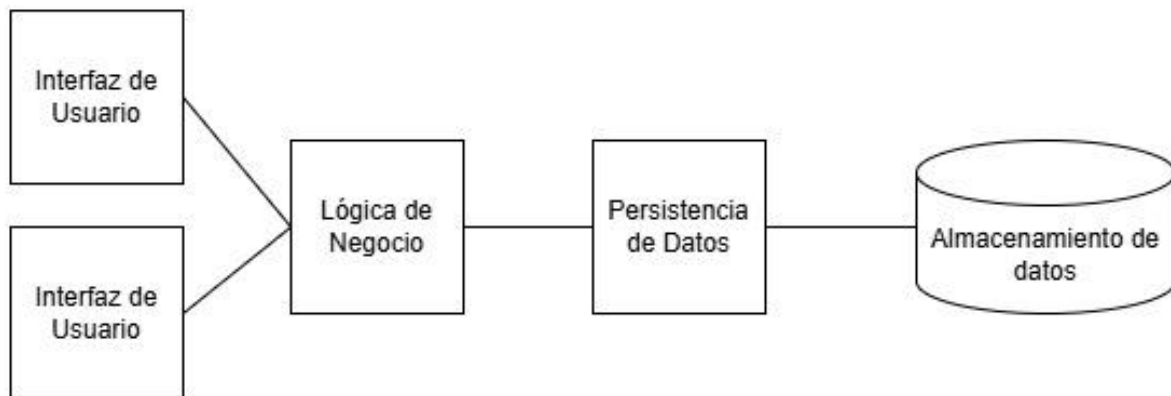


Figura 3. Ejemplo básico una arquitectura multicapa [Elaboración Propia].

4. Microservicios

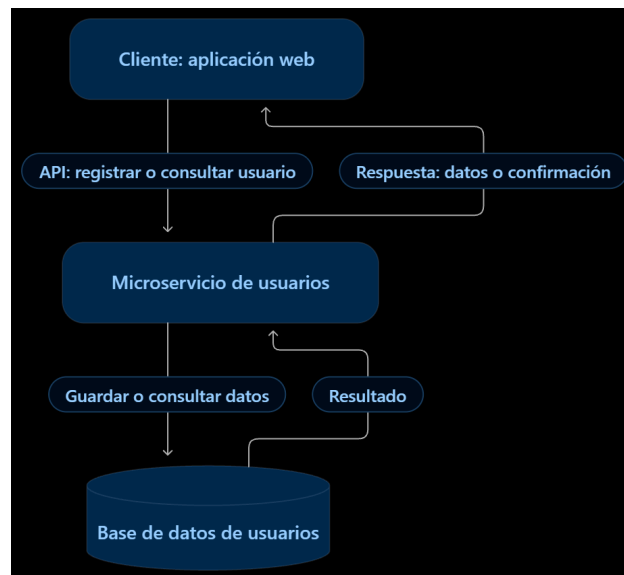


Figura 4. Ejemplo básico de un microservicio de usuarios[Elaboración Propia].

El cliente envía una solicitud al microservicio de usuarios, que se encarga de registrar y consultar usuarios. Para hacerlo, accede a su base de datos y devuelve una respuesta al cliente.

Fase 5: Socialización y matriz comparativa

1. Cada equipo expone su arquitectura durante 5 minutos (proyector)
2. Durante las exposiciones, todos los equipos llenan la matriz comparativa con la información de las cuatro arquitecturas:

GRUPO	Criterio	Cliente-Servidor	Peer-to-Peer	Multicapa	Microservicios
GRUPO 2	Estructura	Protocolos de red http Flujo bidireccional	Red organizada en 4 modelos principales: estructurada, no estructurada, centralizada, híbrida	Capas horizontales con roles específicos y separación por responsabilidades.	La arquitectura se organiza en microservicios independientes, cada uno encargado de una función y de administrar su propia base de datos. Las solicitudes

					ingresan mediante una puerta de enlace o API Gateway , que las dirige al servicio correspondiente. Los servicios se comunican mediante interfaces definidas para intercambiar información.
	Ventajas	Reducción de conflictos Consistencia de datos	Optimización y escalabilidad Tolerancia a fallos	Ventaja de lógica propia de cada nivel, los cambios en una capa no afecta al sistema global.	Los equipos pueden desarrollar y actualizar sus servicios de forma autónoma. Como cada servicio se concentra en una función específica, su código resulta más sencillo de manejar y sus cambios pueden desplegarse con rapidez. Además, se pueden utilizar diferentes tecnologías según las necesidades de cada servicio y organizar las responsabilidades por áreas del negocio.
	Desventajas	La actualización de esta arquitectura se considera un desafío ya que impide mantener conectados a los clientes.	Poca seguridad y control de datos Sincronización de dato	Muchas peticiones de atravesar todas las capas lo que provoca una caída en el rendimiento. Además, existe el riesgo de bloqueo tecnológico ya que para cambiar una tecnología se requiere muchos más recursos que en otras arquitecturas.	La comunicación entre varios servicios puede volverse compleja y generar demoras. También aumenta el trabajo necesario para administrar la aplicación y coordinar los datos repartidos entre distintas bases de datos. Las pruebas y la búsqueda de errores se complican cuando una operación depende de varios servicios.
	Escalabilidad	Favorece la evolución de los sistemas	Alta escalabilidad horizontal	Esta en un nivel intermedio entre monolito y microservicios, pero su extra radica en la separación de responsabilidades.	Puede crecer de forma horizontal, añadiendo más instancias de un servicio para distribuir las solicitudes, o de forma vertical, aumentando los recursos del servidor donde se ejecuta. Esto permite ajustar

					la capacidad de los servicios según su demanda.
	Ejemplos reales	Netflix Youtube Tiendas Digitales	Bitcoin y criptomonedas BitTorrent	ConStore y librerías de Java.	El grupo mencionó Spotify y J. P. Morgan como ejemplos de organizaciones que utilizan microservicios.

GRUPO	Criterio	Cliente-Servidor	Peer-to-Peer	Multicapa	Microservicios
GRUPO 3	Estructura	Medio de conexión red	Los nodos pueden actuar tanto como clientes y servidor	La mas regular incluye 4 capas. De presentación, lógica de negocio, acceso a datos y de datos.	Presentaron una arquitectura con una puerta de entrada o API Gateway, que recibe las solicitudes y las dirige a microservicios independientes. Cada servicio administra su propia base de datos y expone una interfaz para comunicarse con los demás. En su listado también mencionaron un «Puerto de Servicios», aunque ese término necesitaría aclaración para identificar qué componente representa.
	Ventajas	Centraliza información lo cual es facil su mantenimiento	No centralizado Privacidad	Cada capa se maneja individualmente.	Destacaron la autonomía de los equipos: cada uno puede trabajar en sus propios servicios. Al tener responsabilidades delimitadas, los servicios resultan más simples de desarrollar y permiten realizar despliegues rápidos. También señalaron la posibilidad de utilizar distintas tecnologías según las necesidades de cada servicio y organizar la aplicación por subdominios del negocio.
	Desventajas	si el servidor recibe muchas solicitudes puede caer	Los firewall tiende a bloquearlo	En sistemas pequeños se vería un poco innecesario por la complejidad.	Explicaron que la comunicación entre servicios puede volverse compleja o poco eficiente. Administrar muchos servicios aumenta la

					complejidad operativa, mientras que tener bases de datos separadas dificulta coordinar la información. Además, las pruebas y la búsqueda de errores son más complicadas cuando una operación atraviesa varios servicios.
	Escalabilidad	Existen dos formas vertical e horizontal depende mucho del contexto	Alta practicaente infinita	Alta pero para sistemas extensos.	Mencionaron dos formas de crecer: la horizontal, que añade más instancias para repartir las solicitudes, y la vertical, que aumenta los recursos de una instancia o servidor. En microservicios, este crecimiento puede aplicarse a los servicios que necesitan mayor capacidad.
	Ejemplos reales	Banco, cuando se realiza una transferencia	BiTorrent	Se puede usar el ejemplo de sistemas bancarios.	Usaron Amazon para explicar la separación entre Inventario y Pedidos, donde un servicio consulta al otro mediante una API. También usaron Uber para ilustrar la coordinación entre Viajes, Conductores y Cobros, mencionando el patrón Saga para compensar acciones cuando una operación falla. Estos son los ejemplos tal como los planteó el grupo; sus detalles internos requieren una fuente para presentarlos como hechos comprobados.

GRUPO	Criterio	Cliente-Servidor	Peer-to-Peer	Multicapa	Microservicios
GRUPO 4	Estructura	Cliente es quien realiza las solicitudes y el servidor procesa la actividad para dar una respuesta	Red donde cada dispositivo actúa como cliente y servidor al mismo tiempo	Se organiza de forma estructurada por capas.	Presentaron una estructura formada por el frontend, el API Gateway, los microservicios y sus bases de datos. Explicaron que el usuario envía una solicitud desde la interfaz, el gateway la dirige al servicio correspondiente y

					este la procesa utilizando sus propios datos. Al igual que el grupo 3, destacaron la separación de servicios y bases de datos, pero detallaron más el recorrido de la solicitud.
	Ventajas	Desacoplamiento, organización de información	Descentralizado Comunicación directa entre todos los nodos Alta tolerancia a fallos	Alta accesibilidad además de permitir la separación de lógica por capa sin perjudicar a las demás.	Destacaron que cada servicio cumple una función específica y puede actualizarse, probarse y desplegarse de manera independiente. Coincidieron con el grupo 3 en la autonomía de los equipos y el uso de distintas tecnologías. El grupo 4 hizo énfasis en que cada equipo puede asumir la responsabilidad del desarrollo y mantenimiento de su servicio.
	Desventajas	Congestión si no se posee los recursos necesarios	Alta complejidad de software Riesgo de saturación	En sistemas pequeños puede provocar una latencia demasiado elevada.	Coincidieron con el grupo 3 en que la comunicación entre servicios puede volverse compleja y generar demoras. Además, señalaron dos aspectos: el aumento de los puntos que deben protegerse para evitar filtraciones de datos y el consumo acumulado de recursos de todos los servicios.
	Escalabilidad	Incrementar mas recursos.	Teóricamente infinita	Es fácil añadir nuevos nodos por capa ya que no se afecta a las demás, pero si se tiene una estructura muy compleja de capas habrá tardanza en los resultados.	Explicaron que los recursos pueden aumentarse según las necesidades de cada servicio. Mientras el grupo 3 distinguió entre escalabilidad horizontal y vertical, el grupo 4 se centró en destinar capacidad a las funciones que tienen mayor demanda.
	Ejemplos reales	sistema universitario U de Cuenca, ejemplo real es solventar dudas con un docente	Redes de Criptomonedas	Aplicación de Carrito de Compras	Mencionaron Deuna como ejemplo del ámbito financiero y «Delivery» como ejemplo de entrega de comida a domicilio.

					A diferencia del grupo 3, que utilizó Amazon y Uber para explicar la interacción entre servicios, el grupo 4 se enfocó en mencionar aplicaciones de esos dos sectores.
--	--	--	--	--	--

GRUPO	Criterio	Cliente-Servidor	Peer-to-Peer	Multicapa	Microservicios
GRUPO 5	Estructura	cliente el dispositivo capta los output canal red y llega al servidor como un juego en línea	Descentralizado Puede ser de 4 modelos centralizado, estructurado, no estructurado e híbrido		Presentaron servicios independientes organizados por funciones del negocio. A diferencia del grupo 4, se centraron en la división de responsabilidades y no en el recorrido de las solicitudes.
	Ventajas	Pueden dar servicio a varios clientes a la vez	Tolerancia a fallos	Si cae un nodo no se afecta a los demás	Destacaron las actualizaciones independientes, el aislamiento de fallos y el uso de distintas tecnologías. Coincidieron con los grupos anteriores en que equipos pequeños pueden encargarse de servicios específicos.
	Desventajas	Si no se cuenta con características robustas, el servidor no daría respuesta hacia los clientes	Saturación debido a la alta demanda	Riesgo de alta dependencia.	Mencionaron la complejidad operativa, la latencia y la dificultad para mantener los datos consistentes. También explicaron que encontrar errores se complica porque cada servicio tiene sus propios registros.
	Escalabilidad	Mejora de infraestructura	Alta escalabilidad horizontal	Puede extenderse pero aumenta la complejidad.	Enfatizaron la escalabilidad horizontal del servicio con mayor demanda, sin ampliar todo el sistema. Fueron más específicos que el grupo 4 al explicar cómo se aumenta la capacidad.
	Ejemplos reales	Juego en línea.	Windows Delivery Optimization	Una aplicación de comercio electrónico.	Usaron Uber y Spotify para ilustrar la separación de funciones. Coincidieron con el grupo 3 en Uber y con el grupo 2 en Spotify.

GRUPO	Criterio	Cliente-Servidor	Peer-to-Peer	Multicapa	Microservicios
GRUPO 6	Estructura	Se conforma de dos componentes la parte cliente que requiere un servicio y realiza la petición hacia el servidor.	Nodos conectados entre sí, que dividen en partes la información y se comparten la información	Modelo dividido en tres capas.	Presentaron servicios independientes con bases de datos propias. Ampliaron lo expuesto por otros grupos al incluir descubrimiento de servicios, mensajería, orquestación y monitoreo.
	Ventajas	Nuestros dispositivos son parte del cliente, lo cual podemos realizar peticiones al servidor.	Alta tolerancia a fallos	Mantenimiento aislado por capa, si cae una no se afecta a las demás.	Destacaron el despliegue independiente, el aislamiento de fallos y el trabajo paralelo de equipos pequeños, coincidiendo con el grupo 5.
	Desventajas	Si se daña el servidor no hay respuesta	Poco control sobre nodos maliciosos	Complejidad aumentada.	Mencionaron fallos de comunicación y dificultades para coordinar datos y detectar errores. Añadieron la necesidad de trazabilidad distribuida y automatización.
	Escalabilidad	Mejora de infraestructura	Alta escalabilidad horizontal	Es bastante apropiado para ello pero se sacrifica tiempo de respuesta.	Explicaron cómo añadir instancias y repartir solicitudes mediante un balanceador. Complementaron al grupo 5 al mencionar el autoescalado con Kubernetes.
	Ejemplos reales	Servicio Email	BitTorrent	Delivery	Presentaron «Tanque Anti-Derrame», pero el cuadro no detalla cómo aplica los microservicios.

GRUPO	Criterio	Cliente-Servidor	Peer-to-Peer	Multicapa	Microservicios
GRUPO 7	Estructura	Se basa en una estructura de una solicitud de parte del cliente usuario hacia el servidor.	Múltiples nodos conectados entre sí	Partición del sistema en capas.	Presentaron servicios separados por funciones del negocio, conectados mediante API o mensajes. Coincidieron con el grupo 6 en que cada servicio administra sus propios datos.
	Ventajas	Mejor organización, para poder brindar mantenimiento.	Alta escalabilidad y tolerancia a fallos si un nodo cae la red se mantiene	No se pueden saltar niveles entre capas y permite más seguridad.	Destacaron las actualizaciones independientes y el trabajo paralelo de equipos, coincidiendo con el grupo 5.



	Desventajas	Saturación en los servidores, latencia.	Administración complicada Rendimiento variables dependiente de los nodo	Se trabaja como si fuera monolito por lo que si colapsa una capa puede causar una caída en cascada.	Señalaron que coordinar los servicios y mantener los datos consistentes resulta complejo, como explicaron los grupos anteriores.
	Escalabilidad	Mejoramiento de infraestructura	Alta escalabilidad, mientras más nodos mejor	Intermedia ya que si una capa falla las demás también lo que la vuelve ineficiente en cuestión de actualizaciones.	Explicaron que se añaden instancias del servicio con mayor demanda y se reparten las solicitudes, siguiendo lo planteado por el grupo 6.
	Ejemplos reales	Explicación en un ejemplo de un proceso de transferencia en la entidad bancaria del Banco de Loja	Criptomonedas	Tienda universitaria dividida en capa de presentación, de lógica de negocio y la de datos.	Utilizaron una tienda virtual con servicios de Productos, Pedidos y Pagos para mostrar cómo se dividen las responsabilidades.



Universidad
Nacional
de Loja

FEIRNNR - Carrera de
Computación

6. Resultados esperados:

Al finalizar la práctica, el estudiante deberá haber obtenido y documentado lo siguiente:

- Documento de investigación de la arquitectura asignada (definición, componentes, ventajas/desventajas, escalabilidad y 2 ejemplos reales).
- Un diagrama digital de la arquitectura asignada (draw.io o similar).
- Matriz comparativa grupal completa (integrando las cuatro arquitecturas con los aportes de todos los equipos).
- Diseño del caso de estudio: diagrama + justificación técnica (mínimo 3 argumentos) + identificación de transparencia, escalabilidad y tolerancia a fallos.
- Reporte individual breve (máx. 1 página) con conclusiones sobre qué arquitectura considera más versátil y por qué.

7. Preguntas de Control:

- **¿Qué es un sistema distribuido y cuáles son sus características fundamentales que lo diferencian de un sistema centralizado?**

Un sistema distribuido son nodos independientes conectados para funcionar como un único sistema opera mediante concurrencia entre múltiples nodos carece de un reloj global unificado, tolera fallos parciales redistribuyendo la carga de trabajo y logra una alta disponibilidad mediante el escalado horizontal, en cambio un sistema centralizado depende de un único nodo principal que procesa y almacena todas sus operaciones lo que la vuelve vulnerable a caídas totales y limita su crecimiento al escalado vertical mediante hardware más costoso

- **Explique la diferencia de comunicación entre la arquitectura cliente-servidor y la arquitectura peer-to-peer.**

Mientras que en cliente-servidor el flujo es asimétrico y centralizado los clientes envían peticiones a un servidor central que procesa y responde obligatoriamente como intermediario lo que genera posibles cuellos de botella y puntos de fallo en cambio en peer-to-peer el flujo es simétrico directo y descentralizado ya que todos los nodos poseen el mismo rango y funcionan simultáneamente como clientes y servidores permitiendo la transferencia directa entre usuarios.

- **¿Qué rol cumple el middleware en un sistema distribuido y en qué capa se ubica?**

El middleware es el encargado de ofrecer conexiones persistentes entre la red o el sistema operativo y las aplicaciones que necesitan del mismo para ejecutar sus acciones. De esta manera, se puede tener varias aplicaciones o corriendo en servidores, redes o sistemas operativos diferentes, pero que funcionen como si fueran una sola. Además permite ocultar la complejidad de la infraestructura, las diferencias de hardware y los distintos lenguajes de programación.

- **En la arquitectura de tres capas (3-tier), ¿qué función cumple cada capa y por qué separarlas mejora el mantenimiento?**

En esta arquitectura se tiene la capa de interfaz de usuario, para interactuar directamente con los clientes y recibir sus peticiones. Seguido se encuentra la capa de negocio en donde se concentran todos los algoritmos y reglas de negocio particulares del sistema. Finalmente, está la capa de persistencia de datos conformada por una base de datos que

se encarga de almacenar toda la información en esquemas claros y estructurados.

- **¿Cuáles son las principales ventajas de los microservicios frente a un diseño monolítico, y qué desafíos introducen?**

La principal ventaja de los microservicios es que cada servicio se puede modificar, desplegar y ampliar por separado. En un diseño monolítico, un cambio puede exigir desplegar toda la aplicación; con microservicios, podemos actualizar únicamente la función que necesita ese cambio. También permiten que distintos equipos trabajen en servicios específicos y que se aumente la capacidad de los más utilizados. Además, si los fallos están bien aislados, un servicio puede fallar sin detener toda la aplicación.

El desafío es que esas partes necesitan comunicarse por la red, lo que puede generar demoras y fallos de comunicación. También se vuelve más complicado mantener consistentes los datos repartidos entre servicios y encontrar el origen de un error. Por eso, hay que comprobar tanto que cada servicio funcione por separado como que todos trabajen correctamente juntos.

- **¿Qué arquitectura consideraría más adecuada para una aplicación como BitTorrent y por qué?**

La arquitectura más adecuada sería la peer-to-peer ya que al dividir los archivos en pequeños fragmentos cada nodo que descarga partes de un archivo las comparte inmediatamente con los demás logrando que el ancho de banda total y la velocidad de la red escalen positivamente a mayor número de participantes reduciendo drásticamente los costos de servidores centrales y eliminando puntos de fallo al permitir que la descarga continúe de forma resiliente aunque el creador original se desconecte.

- **¿Cómo influye la escalabilidad horizontal y vertical en la elección de una arquitectura distribuida?**

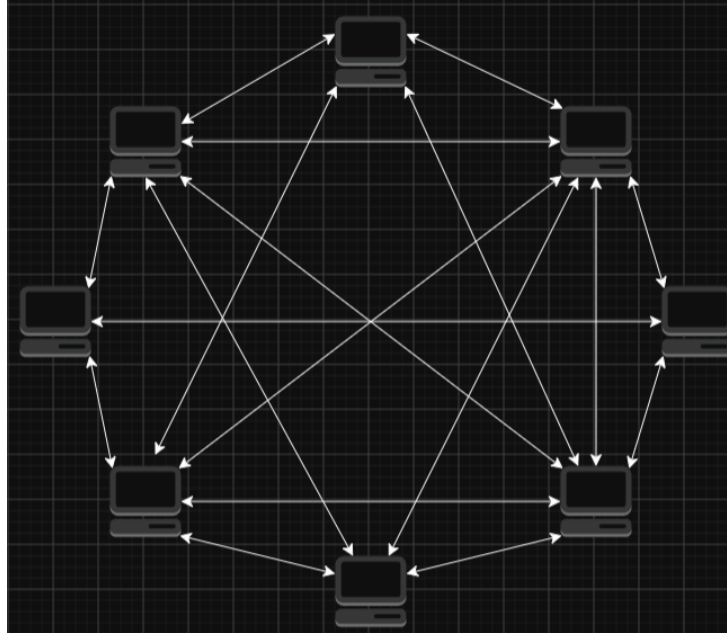
Influye en cómo esperamos que crezca el sistema. La escalabilidad vertical consiste en darle más capacidad a una máquina, como aumentar su memoria RAM o mejorar su procesador. Es más sencilla de aplicar, pero llega un punto en que ya no podemos seguir ampliándola. La horizontal consiste en añadir más máquinas y repartir el trabajo entre ellas. Por eso, si esperamos muchos usuarios o un crecimiento constante, conviene elegir una arquitectura que permita distribuir la carga. Eso sí, también tendremos que coordinar esas máquinas para que trabajen correctamente.

- **¿Qué significa transparencia en un sistema distribuido y mencione dos tipos de transparencia?**

La transparencia significa que el usuario puede utilizar el sistema sin tener que conocer cómo están organizadas las máquinas que hay detrás. Aunque participen varios equipos, para él funciona como un solo sistema. Dos tipos son la transparencia de ubicación, que permite acceder a un recurso sin saber en qué máquina está guardado, y la transparencia de acceso, que permite utilizar un recurso de una forma similar tanto si está en nuestro equipo como si está en otro conectado a la red.

- **Si un servidor central falla, ¿qué arquitectura continúa funcionando y por qué?**

La arquitectura que continuará funcionando sería la Peer to Peer porque no depende de un nodo central, cualquiera de los nodos que están interconectados podría funcionar tanto como cliente o como servidor.



- **En su caso de estudio, ¿qué criterios fueron decisivos para elegir la arquitectura propuesta?**
 - El sistema es resistente a fallos debido a que no se depende de un nodo maestro como en el caso de cliente-servidor por lo que se puede tener distribuidos los servicios haciéndolo mucho más seguro frente a fallos eléctricos o en los servidores.
 - Este tipo de arquitectura no muestra los fallos directamente a diferencia de los microservicios en donde si un servicio falla se muestra inmediatamente en el sistema.
 - Presenta una alta escalabilidad ya que se pueden añadir nodos progresivamente.

8. Conclusiones

- **Arellys Ajila**

Es importante la comparativa directa entre arquitecturas para entender las diferencias clave entre ellas debido a que suelen parecer iguales entre sí pero de este modo se puede abstraer las características principales de cada una.
- **Josue Asanza**

Entender las arquitecturas nos permite que al analizar los requerimiento y contextos de nuestro proyecto podamos hacer la mejor elección de arquitectura , sabiendo tanto su ventajas desventajas y las opciones que tenemos para escalar y mantener nuestra red
- **Wagner Balcazar**

Al Investigar y comprender acerca de cada una de las arquitecturas, nos ayuda a entender de que arquitectura se adaptaría de mejor manera a los diferentes contextos de proyecto, y además conocer en donde se presentan en aplicaciones ya existentes y al asociarlas con ejemplos reales fue de gran ayuda para la comprensión de conceptos.
- **Douglas Carreño**

Esta práctica me ayudó a entender que la forma de organizar una

aplicación depende de lo que necesita y de cuánto puede crecer. Me parecen útiles los microservicios porque permiten cambiar o mejorar una parte sin tener que modificar todo, aunque hay que cuidar cómo se comunican y comparten la información. Como futuro desarrollador, aprendí que no basta con dividir una aplicación en partes: cada una debe tener una función clara y trabajar bien con las demás.

9. Evaluación

No aplica en esta práctica

10. Bibliografía

[1] M. Lim, "C2CFTP: Direct and Indirect File Transfer Protocols Between Clients in Client-Server Architecture," in *IEEE Access*, vol. 8, pp. 104278-104290, 2020, doi: 10.1109/ACCESS.2020.2998725^[1].

[2] D. Castellà Martínez, "Computación Distribuida en Entornos Peer-to-Peer con Calidad de Servicio," Tesis doctoral, Dept. d'Informàtica i Enginyeria Industrial, Universitat de Lleida, Lleida, España, mayo 2011

[3] T. Lynn, J. G. Mooney, B. Lee, and P. T. Endo, Eds., *The Cloud-to-Thing Continuum: Opportunities and Challenges in Cloud, Fog and Edge Computing*, 1st ed. Cham, Switzerland: Palgrave Macmillan, 2020, doi: 10.1007/978-3-030-41110-7.

[4] N. Alshuqayran, N. Ali, and R. Evans, "A systematic mapping study in microservice architecture," in *2016 IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA)*, 2016, pp. 44–51, doi: 10.1109/SOCA.2016.15.

[5] V. Velepucha and P. Flores, "A survey on microservices architecture: Principles, patterns and migration challenges," *IEEE Access*, vol. 11, pp. 88339–88358, 2023, doi: 10.1109/ACCESS.2023.3305687.

[6] M. Waseem, P. Liang, G. Márquez, and A. Di Salle, "Testing microservices architecture-based applications: A systematic mapping study," in *2020 27th Asia-Pacific Software Engineering Conference (APSEC)*, 2020, pp. 119–128, doi: 10.1109/APSEC51365.2020.00020.

11. Elaboración y Aprobación

Elaborado por	José Guamán Docente	
Ejecutado Por	Rene Rivera Técnico Docente	
Aprobado por	Edison L Coronel Romero Director de Carrera	



Universidad
Nacional
de Loja

FEIRNNR - Carrera de
Computación

12. Declaración de IA

Dentro de esta actividad se utilizó la herramienta de IA (Nootebooklm) con la finalidad de obtener información acerca de los temas correspondientes, basándose en bibliografía otorgada por cada uno de los integrantes del grupo, así mismo como evidencia se deja los prompts utilizados:

puedes darme la referencia del archivo que te mande en IEEE

Analiza el artículo adjunto y dame la en cita IEEE, y definición acerca de la arquitectura cliente-servidor.